

System Architecture & Deployment Topology/Arrangement

The AEGIS system uses a Distributed Cloud Architecture by exploiting production-ready free-tier platforms. This architecture isolates the presentation layer, application orchestration backend, data persistence, and untrusted code execution sandbox to ensure decoupling and modularity.

Component Selection & Comparative Analysis

Frontend Hosting: Vercel

- **Approach:** Serverless deployment by using Vercel's **Edge Network**.
- **Alternatives Considered:** Self-hosting on a traditional Linux server, AWS S3 and CloudFront.
 - **Justification vs. Alternatives:** AWS S3 and CloudFront offer enterprise scaling; their configuration carries variable data-eegress financial risks. Vercel serves static web assets through a global Content Delivery Network (CDN) straight out of the box without any cost. This guarantees fast initial page loads globally.
- **Non-Functional Requirement Alignment:**
 - **Scalability & Concurrency:** The CDN design can handle traffic spikes up to and beyond 500 concurrent users without UI latency.
 - **Maintainability:** Inherit GitHub integration triggers automatic compilation and deployment upon code merges, meeting the required **2-hour deployment window requirement**.

Commented [1]: (a globally distributed CDN and compute platform that serves your web content and executes code from Points of Presence (PoPs) closest to your users, drastically reducing load times)

Backend Orchestration Layer: FastAPI and Piston on Render

- **Approach:** Containerised deployment of the harmonized application backend and code execution sandbox environment on Render's managed Cloud Application Platform.
- **Alternatives Considered:** Koyeb (Deprecated Free Tier), Self-Hosted Oracle Cloud Infrastructure (OCI) Dedicated Virtual Machine
 - **Justification vs. Alternatives:** Since Koyeb's free-tier structure was deprecated, Render was selected to maintain a zero-budget layout without introducing massive DevOps complexity of self-hosted single-node container orchestration or reverse proxy configuration. While a dedicated virtual machine avoids resource constraints, it demands cumbersome maintenance and custom script automation. Render shifts the operational complexity onto a fully managed infrastructure,

accepting specific, predictable performance trade-offs in exchange for continuous deployment automation.

▪ **Non-Functional Requirement Alignments & Trade-offs:**

- **Performance:** To respect the budget constraints, the backend accepts Render's free tier sleep mechanism, which suspends the container instance after 15 minutes of inbound inactivity. Therefore, the 2-second target response time applies to an active instance; initial requests will encounter a 30-to-50 second cold-start delay.
- **Concurrency & Scalability:** The host cluster imposes a strict 512MB RAM resource ceiling. To handle concurrent user actions without prompting Out-Of-Memory (OOM) kernel crashes, the runtime application relies on FastAPI's cooperative asynchronous event-loop architecture to maximise processing efficiency under limited memory allocation
- **Resiliency:** System recovery bypasses fragile custom scripting by depending on Render's native deployment history mechanisms. In the event of an unexpected runtime failure or unstable deployment, rollbacks are executed through the platform's control plane to instantly reroute production traffic back to the cached, last-known-stable container artifact. This yields a recovery execution. Time of under 60 seconds

Data Persistence Layer: Supabase (Managed PostgreSQL)

- **Approach:** Cloud-managed PostgreSQL instance by Supabase.
- **Alternatives Considered:** Self-hosted PostgreSQL via Docker, SQLite.
 - **Justification vs. Alternatives:** SQLite does not have the required concurrent write capabilities. Self-hosting PostgreSQL on the free tier heavily consumes memory, risking Out-Of-Memory (OOM) crashes. Supabase provides an isolated database environment at no cost.
- **Non-Functional Requirement Alignment:**
 - **Security:** Natively, Supabase manages **AES-256 encryption at rest and SSL encryption in transit**, satisfying data privacy requirements.
 - **Scalability & Concurrency:** Includes Supervisor for database connection pooling, preventing connection exhaustion when 500 concurrent users execute database queries.

Commented [2]: Supabase automatically handles securing your data across its entire lifecycle without requiring any setup or configuration on your part.

Commented [3]: Supabase has a built-in traffic cop (called Supavisor) that stops your database from crashing when hundreds of users use your app at the exact same time

Non-Functional Requirements Traceability Matrix

NFR Identifier	Quality Requirement	Architectural Decision	Evaluation Status
Performance: NFR-01	Response time <= 2 seconds	Delegation of application hosting to Render's managed runtime. Acknowledging a free-tier constraint, the quality requirement latency target applies exclusively to active instances.	N/A
Concurrency: NFR-02	Support for 500+ concurrent users	Separation of static asset delivery to the Vercel Global Edge CDN, coupled with the asynchronous event loop design of FastAPI running within Render's 512MB RAM allocation container boundary.	N/A
Security: NFR-03	Secure execution of untrusted code	Containerized code execution via Piston, relying on Render's network shielding and platform-enforced AES-256 cryptographic encryption at rest through managed Supabase PostgreSQL.	N/A
Reliability: NFR-04	Availability >= 99.5% uptime	Deployment on Render's managed cloud infrastructure to leverage automated platform health monitoring, accepting free-tier monthly account sleep cycles while eliminating self-managed server maintenance overhead.	N/A
Maintainability: NFR-05	Automated build verification and deployment < 2 hours	Configuration of Git-driven automated CI/CD hooks directly integrated with production main-line branch merges.	N/A

Commented [LN4]: Need to revisit to formulate an execution plan.

Operational Deployment Specifications

Environment Parity Configuration

The system isolates operational contexts using Git branch tracking:

- **Development Environment:** Configured locally via the team's individual workstations by using Docker Compose containers pointing to a separate Supabase local/dev schema.
- **Production Environment:** Triggered automatically via GitHub Webhooks. Any pull requests merged into the main branch initiate an immediate secure cloud compilation and update the live endpoints on Vercel and Koyeb with no manual intervention.

Infrastructure as Code & Reproducibility

The deployment ecosystem is documented and defined via a root-level **docker-compose.yml** file to prevent configuration drift and satisfy reproducibility.

Since Piston does not make use of additional tracking databases, the configuration remains simple, allowing any evaluator to clone the repository and run a local mirror with a single command: **docker compose up --build**.

Secrets Management Protocol

Credentials, connection strings, and cryptographic keys are strictly excluded from version control tracking:

- An explicit template file (**.env.example**) is maintained publicly in the root directory to specify structural dependencies.
- The production **.env** configuration file is directly attached to the system's **.gitignore** rules.
- Production keys are injected into application memory at runtime through Vercel and Koyeb's secure Web UI Environment Panels, ensuring that no plain-text storage exists in code repositories.

Automated Rollback Engineering

In the event of an unexpected failure or unstable deployment during runtime evaluations, a repeatable rollback strategy is utilized:

- **Edge Layer (Frontend - Vercel):** Vercel's Dashboard retains immutable historical build images tied directly to their respective Git commit hashes. A rollback is executed instantly by designating a previous stable SHA as the active production target (recovery time: < 10 seconds). Thus, our system utilizes Vercel's native **Image Tag Pinning**.
- **Application Layer (Backend – Piston & Render):** Render's native build history is leveraged as rollbacks are executed via Render's control plane by instantly rerouting production traffic back to the cached, last-known-stable container image artifact

containing both the FastAPI logic and Piston engine (recovery time < 60 seconds). Thus, our system utilizes Render's native **Immutable Artifact Rollback**.

Commented [LN5]: The strategies are the same, but what makes them different is the artifact being restored.

Rejection of Industry-Standard Hyper-Scale Clouds (AWS, GCP, Azure)

Hyper-scale cloud providers such as Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure are industry standards for enterprise-grade applications; they were thoroughly evaluated and rejected for this project. The decision to use platforms is rooted in a direct conflict between their structural limitations and the project's non-functional requirements (NFRs).

As a team, we identified three core dimensions where hyper-scalers failed to meet project constraints:

Financial Volatility

- **The Constraint:** The project works under a strict R0/month budget threshold.
- **The Hyper-Scaler Failure:** AWS, GCP, and Azure operate on utility-based, pay-as-you-go pricing models. While they offer initial "Free Tiers," these tiers include hidden variable costs, mainly **Data Egress Fees** (charges for data leaving the cloud network).
- **Concurrency (NFR-02) Impact:** Under a stress-test scenario, simulating **500 concurrent users** continuously sending code payloads, streaming stdout/stderr buffers, and querying databases, the network traffic would quickly breach free data transfer caps. A minor configuration oversight or a malicious script could trigger automated scaling mechanisms, resulting in catastrophic, unbudgeted financial liabilities ("**bill shock**"). The selected stack (Vercel, Koyeb, Supabase, Oracle) hard-caps resource usage at the free limit, guaranteeing zero financial risk.

Severe Resource Starvation on Free Tiers

- **The Constraint:** The system must compile and execute multi-language user source code securely and return execution states in under 2 seconds.
- **The Hyper-Scaler Failure:** The compute instances provided under hyper-scaler free-tiers - such as the **AWS t2.micro/t3.micro** or **GCP e2-micro** - are very restricted. These machines provision only **1 vCPU and 1GB of RAM**, operating on a "burstable" CPU credit model.
- **Performance (NFR-01) & Security (NFR-03) Impact:** Code compilation (mainly for typed languages like Java or C++) is intensely CPU-bound. Forcing an application backend and a containerized execution engine onto a **1GB RAM** instance would cause **immediate kernel Out-of-Memory (OOM)** crashes under concurrent load. In addition, once burst credits are exhausted, the CPU is heavily throttled, destroying the target **< 2 second response time**. In contrast, Oracle Cloud's Always-Free tier

grants access to 4 dedicated ARM vCPUs and 24GB of RAM, providing the physical hardware required to run Piston smoothly at scale.

Excessive Operational Complexity and Overhead

- **The Constraint:** The deployment must be highly maintainable, fully automated, and reproducible under a tight 2-hour implementation window.
- **The Hyper-Scaler Failure:** Architecting a containerized technical assessment platform on AWS requires manual configuration or heavy scripting for Virtual Private Clouds (VPCs), Identity and Access Management (IAM) security policies, Subnets, Internet Gateways, and Elastic Container Service (ECS) cluster definitions.
- **Maintainability (NFR-05) Impact:** This “Click-Ops” or extensive Infrastructure-as-Code (IaC) overhead introduces significant configuration friction. For a small development team, managing cloud-provider-specific network security patches takes time away from product feature development. Platforms like Vercel and Koyeb abstract this infrastructure layer entirely through native GitOps pipelines, allowing the team to meet the **2-hour automated deployment window with** zero infrastructure configuration maintenance.